

CS 505, Part I: Security issues in Databases

Victor W. Marek¹

Department of Computer Science
University of Kentucky

Spring 2014

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

¹This is copyrighted material, available *only* for registered students of CS 505. Spring 2014

- ▶ For instance companies collect financial information, computing economic trends, time series, other information valuable for themselves and their clients
- ▶ If such information, often unique and collected over time, is divulged, the company may lose significant assets in services that are no longer needed
- ▶ So, the information needs to be protected
- ▶ This part of our lecture is about techniques for such protection
- ▶ Of course, we will only touch the “tip of the iceberg”

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Within an organization (company, office) one may quite often have various categories of users
- ▶ For instance, in financial industry, one often needs to institute *Chinese wall* between analysts that work for the company itself, and analysts providing services to clients
- ▶ Thus there are natural “horizontal” partitions of people in the business
- ▶ But there are also “vertical” partitions; some people may have more information than others, or even incomparable amounts of information

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Let us look at the doctor's office

- ▶ Who should have the access to my medical record?
- ▶ Any person in the office?
- ▶ When they have the access, could they read, write, update?
- ▶ Should the information be encrypted? If so, how?
- ▶ Should I have the access to my medical information, in particular doctor's notes?
- ▶ Should doctors share the information about patients?
- ▶ If so, should it be voluntary?
- ▶ How could we support sharing, esp. in view of the health reform?
- ▶ Standards for sharing?
- ▶ Security requirements?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Lab test results?
- ▶ (Are the security standards on access same as in the office that asks for tests?)
- ▶ Referring and referred physicians - what about the security requirements there?
- ▶ Hospitals - analogous issues
- ▶ Insurers - what kind of information is shared with them?
- ▶ Providers of pharmaceuticals - ditto

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ When a large legal practice represents a client it may run into *conflict of interest* issues
- ▶ If such legal office represents a client (individual or a business) the individuals involved can not (at present or in the past) represent its adversary (adversaries)
- ▶ Legal offices are, often, retained for representation of companies
- ▶ If a potential client needs representation in a legal conflict with some entity that has the company on retainer, then there is a potential conflict of interest

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Worse, *past* legal cases may contain information affecting current ones
- ▶ Access to such information may be illegal
- ▶ So, what would be a model of access to information within law offices?
- ▶ (Is there only one model?)
- ▶ Should some form of *Chinese Wall* be instituted there as well?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartments mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Often even possession of information that have been acquired from unknown sources may result in *patent infringement* or *trade secrets infringements*
- ▶ Because when such information is available, conscious or unconscious violation of patents or *trade secrets* may occur
- ▶ The issue, then, is how to prevent such situation
- ▶ Even when the information is not patented, its unauthorized use can be infringement on rights of inventors

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What about software companies?

CS 505- Spring 2014

- ▶ What software elements can be protected?
- ▶ I visit some software company, they tell me about a software they plan to produce. What then?
- ▶ Nondisclosure agreements
- ▶ The issue of licenses
- ▶ (The story of SCO and Linux)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Of course military and other branches of government have secrets
- ▶ Often information is classified and requires clearances
- ▶ But government secrets go well beyond military
- ▶ Knowing plans of, say, Treasury Department or of Federal Deposit Insurance Corporation may result in significant financial gains (because when a bank is seized its shares loose value). Thus selling shares *short* may benefit an individual or an institution that has the access to such information.
- ▶ Law forbids the use of such knowledge
- ▶ Another example: knowing the judgement of Federal Drug Administration on specific drugs may result in significant gains/losses (Martha Stewart affair)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn in this section?

CS 505- Spring 2014

- ▶ Information is a valuable commodity and needs to be protected
- ▶ When this information is stored in database the access to database information needs protection
- ▶ Essentially all areas of human activity deal with information and such areas may have different views of what constitutes security
- ▶ It is unlikely that there is a “one-size-fits-all” approach

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Often the access to data is not related to the specific user but to the position that user has in a company
- ▶ For instance every teller in a bank may edit customers records
- ▶ Every customer service representative may access all customers records etc.
- ▶ In both examples the access rights are not related to the specific individual, but to the *role* s/he plays in the business process
- ▶ We need a framework, esp. in business situations, to accommodate roles

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Let us assume that the user *marek* has the exclusive access to personal records of *sales* department
- ▶ Now, *marek* takes his annual leave
- ▶ *marek*'s deputy, *dexter*, should have the access to these records during *marek*'s absence
- ▶ What mechanism could be used to make sure this happens seamlessly?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ (For a moment let us not look at roles, we will discuss them later)
- ▶ Then, we have three categories of objects in security:
 - ▶ Subjects (users, users' programs)
 - ▶ Objects (it is unclear what granularity of objects we select - it could be specific databases, specific tables, specific records, or even columns in records)
 - ▶ Privileges (read, write, update, possibly also *grant* (to implement delegation))

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Once we accept such approach we can think about a three-dimensional matrix, with Boolean entries
- ▶ This matrix can be implemented in more than one way. For instance, we can implement it by means of *control access lists*
- ▶ Specifically, we can assign to an object a list consisting of records of the form (*subject*, *privilege*)
- ▶ Then, for instance, to see if *marek* can update the table *employees*, we check if the access list for the table *employees* includes an item (record): (*marek*, *update*)
- ▶ This general access model *conceptually* relates to access lists, but for a specific access model we may have other mechanisms for implementation

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ The issue of granularity may seem to be unimportant
- ▶ But in reality it is very important in business applications
- ▶ To give an example, the personal records of employees can be viewed by all the customer service representatives in HR department
- ▶ Such representatives should be able to change, say, employee's phone number or address
- ▶ But changing the field *SSN* must be seriously restricted, and likewise for *salary* field

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Who controls the access to data?

CS 505- Spring 2014

- ▶ In business applications we want flexibility
- ▶ This is usually called *Discretionary Access Control* and is implemented by all major SQL-based DBMSes
- ▶ In other applications we may want a tight, centralized control. This is usually called *Mandatory Access Control* and is not part of SQL standard (but likely will be)
 - ▶ We may want to protect secrets (this is called *Bell-La Padula* model).
 - ▶ We may want to make the information more authoritative (this is called *Biba model*)
- ▶ There are other models of security, for instance *Chinese Wall* (also called Brewer-Nash model) - related to our law example, but also to financial institutions

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn?

CS 505- Spring 2014

- ▶ There is a general access model
- ▶ There are variations (for instance related to the issue of who controls modification of access rights)
- ▶ These variations may have various implementations

Introduction

Few issues in
business, an aside

Access to data

Simple protection
mechanism - the
views

Access control in
SQL

Mandatory Access
Control

Compartment
mechanism, and
other access
models

Role-based
access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Relational views are often used as a security mechanism
- ▶ As we recall from CS405 (or similar), a view defines a (virtual) table
- ▶ We can use such tables to define what a user can and what she can not see
- ▶ All versions of SQL support view definitions, and SQL provides means to define views
- ▶ Recall that the syntax for creating views is:
CREATE VIEW *viewName*(Att_List)
AS *query*
- ▶ Here *query* is a valid SQL query
- ▶ (Att_list is optional, and can be used for renaming)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

An example

- ▶ A retirement community maintains a database of its live_in clients
- ▶ We have a scheme *sen_cit*(lname, fname, y_o_b, ssn, room_no)
- ▶ But we want the receptionist to see only last name, first name and room number (for she has no *need to know* the year of birth, or ssn)
- ▶ We create a view:

```
CREATE VIEW seniors  
AS  SELECT lname,  fname,   room_no  
    FROM sen_cit
```
- ▶ We give the receptionists the right to read (i.e. `SELECT * FROM seniors`) this view

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

More on views

- ▶ Generally, user does not know if the table she processes is a stored table, or a view
- ▶ This creates problems, here is an example. The retirement community creates a list of persons who will attend some social function. This list is created as a view. But now we want to invite someone to join that function, and this requires that we insert his personal data to the table *seniors*
- ▶ This person, *marek, victor* is *not* in the table *sen_cit*. We still would like to have him in the table.
- ▶ But the view *seniors* is **not stored** in the database; it is **derived** from the table *sen_cit* (which is stored)
- ▶ We can, actually, update the view *seniors*.
- ▶ We can insert into the table *sen_cit* the record

(*marek, victor*, NULL, NULL, NULL)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Not every view can be updated
- ▶ Generally we can not update any view that is not “simple”
- ▶ Here are the limitations:
 - ▶ We can update fields coming from only one table - in practice the view needs to be on a single relation
 - ▶ No aggregates, no DISTINCT in the definition
 - ▶ Attributes *not* listed in SELECT clause can not be NOT NULL, or the part of the PRIMARY KEY of the table from which the view is derived
 - ▶ (Why?)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Funny things about views

CS 505- Spring 2014

- ▶ You may try to insert into a view a tuple that does not satisfies view definition
- ▶ Ex: We again define a view similarly to one before:

```
CREATE VIEW A_seniors
AS  SELECT lname,  fname,   room_no
      FROM sen_cit
      WHERE lname LIKE A*
```
- ▶ These are the records of individuals whose last name starts with "a"
- ▶ Now try to insert (*Marek, Victor, NULL*) which does *not* satisfy the WHERE clause
- ▶ To provide control in such situation SQL provides the appropriate language: CREATE VIEW has optional WITH CHECK OPTION qualifier

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ When a view is no longer needed it can be dropped
- ▶ `DROP VIEW view_name [RESTRICT|CASCADE]`
- ▶ `RESTRICT` destroys the view *view_name* **providing** the name is not used in other definitions (of a view, trigger, assertion (i.e. an integrity constraint), SQL routine) but, of course, does not affect the tables stored in the database. Error message if dependencies exist
- ▶ `CASCADE` destroys all the views, triggers, assertions, etc. dependent on *view_name*

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn?

CS 505- Spring 2014

- ▶ To support the “need-to-know” paradigm (you get data needed for your job but not more) SQL provides the notion of a view
- ▶ Views are like tables, but they live ‘virtually’, we store their definitions, not their extents
- ▶ Views are often sufficient to control access to information
- ▶ There are problems with updating views, some issues are related to view definitions (Ex: views using multiple tables), some with their semantics

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Discretionary Access Control (DAC)

- ▶ We will discuss it at the granularity level of tables (but finer levels of granularity are available in RDBMSes)
- ▶ We will also disregard the temporal aspects of access control
 - ▶ But these aspects are very important; we should be worried if, for instance, someone accesses the table *clients* outside of working hours of the bank (but look at Auditing section)
- ▶ In this model the privileges are: select (i.e. read), write, update, drop, grant (actually, there are others)
- ▶ When the user *marek* creates the table *clients* he is granted all privileges on that table
- ▶ Thus *marek* can execute SQL queries on the table *clients*
- ▶ A delicate aspect: What about seeing the schema of the table. Surely, *marek* can view it. What about the others?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Who can access the table *clients*?

- ▶ Surely, *marek* can access it.
- ▶ But also the *database administrator* (DBA) can access it.
- ▶ Thus the position of a DBA involves quite a lot of trust
- ▶ (This points to the fact that the dangers in loosing control over data may come, and in fact mostly come, from *insiders*)
- ▶ Unless we change the control access list for the table *clients* the user *dexter* will not be able to see, query, modify, reference, etc. table *clients*.
- ▶ Where is access control table?
- ▶ How is access control table changed?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Access control table

- ▶ It is, in fact, an ordinary database table (in addition to fields privilege and user it may have other fields, for instance time stamp, for audit purposes)
- ▶ It just is not visible (depending on the database set up) to all users.
- ▶ This table is one of *system tables*
- ▶ SQL provides means for updating this table by means of GRANT and REVOKE commands.
- ▶ In effect, GRANT allows for adding privileges, REVOKE eliminates privileges.
- ▶ (There are, as we will see, delicate aspects of these commands)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ SQL allows for transferring privileges; when one user grants a privilege (on a table, or on a view) she can qualify granting by WITH GRANT OPTION
- ▶ *marek* is the owner of the table *clients*
- ▶ But now *marek* takes the annual leave and *dexter* takes over
- ▶ First, *marek* needs to give permissions to *dexter* on the table *clients*
- ▶ Those have to be explicitly stated. Say *marek* wants to give *dexter* reading and inserting privileges on *client*
- ▶ GRANT SELECT, INSERT ON *clients* TO *dexter*
- ▶ We can also write two lines:
 - ▶ GRANT INSERT ON *clients* TO *dexter*
 - ▶ GRANT SELECT ON *clients* TO *dexter*

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What can dexter do?

- ▶ *dexter* can both read the data from table *clients* and insert into *clients*. But he can not delete, update, or drop
- ▶ This, of course provides additional security to the table *clients*
- ▶ Granularity of grants can be changed (e.g. *marek* may allow *dexter* update only some fields)
- ▶ However this syntax does not allow *dexter* further grants

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ SQL has language for permitting further grants
- ▶ Example: GRANT SELECT, INSERT ON *clients* TO *dexter* WITH GRANT OPTION
- ▶ Now *dexter* can allow *alice* to read the table *clients*
- ▶ Afterwards *alice* can read table *clients*
- ▶ But *alice* can not insert into the table *clients*
- ▶ Actually, *dexter* may allow *alice* to read *clients* WITH GRANT OPTION and if *alice* now grants reading privilege to *bob* then *bob* will be able to read the table *clients*
- ▶ But what if *dexter* gives *alice* reading privilege (SELECT) without grant option?
- ▶ Actually, nothing will prevent *alice* to grant both SELECT and INSERT privileges on *clients* to *bob* with grant option
- ▶ Will *bob* be able to insert into *clients*?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

But *marek*, eventually, comes back ...

- ▶ Of course, first thing he does, he revokes privileges of *dexter* on the table *clients*
- ▶ We need a language for such revoking operations
- ▶ SQL provides the language for revoking privileges
- ▶ `REVOKE privilege_list FROM grantee [RESTRICT | CASCADE]`
- ▶ In our case: `REVOKE SELECT INSERT on clients FROM dexter CASCADE`
revokes the two privileges granted by *marek* to *dexter* **and** also the subsequent privileges that *dexter* and his crew distributed

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ What about RESTRICT option (which, actually, is default)?
- ▶ It protects against unintended consequences. If *dexter* gave further privileges to other individuals (or roles), an error message results
- ▶ In our case this is what would happen

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Graph representation of permissions

CS 505- Spring 2014

- ▶ Actually, a multigraph
- ▶ (Recall our restriction on the granularity; we talk about entire table, not individual records, and not individual fields)
- ▶ The graph $G_{clients}$ is assigned to the table *clients*
- ▶ (Of course the construction is general, for each table we have a graph)
- ▶ The nodes are users; in our example *marek*, *dexter*, *alice*, *bob*, etc.
- ▶ We add an additional node that we will call *system*
- ▶ The edges are always labeled with permissions and an additional bit denoting presence or absence of WITH GRANT OPTION
- ▶ In our example, when *marek* created the table *clients*, we created edges from *system* to *marek*

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanisms, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

The graph of permissions, cont'd

CS 505- Spring 2014

- ▶ This initial edges are from *system* to *marek*, and they are labeled $\langle \textit{permission}, 1 \rangle$ where *permission* ranges through: SELECT, INSERT, DELETE, UPDATE, DROP (all WITH GRANT OPTION)
- ▶ Now, when *marek* gives *dexter* the permissions to SELECT and to INSERT, with grant option, we create two more edges in $G_{clients}$
- ▶ One is $\langle \textit{marek}, \textit{dexter}, \text{SELECT}, 1 \rangle$ and the other $\langle \textit{marek}, \textit{dexter}, \text{INSERT}, 1 \rangle$
- ▶ But edge in our graph is just a record:

$\langle \textit{grantor}, \textit{grantee}, \textit{privilege}, \textit{grant_bit} \rangle$

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

The graph G_{table} , cont'd

- ▶ When *dexter* gave *alice* SELECT privilege on *clients* without grant option, then we created the following edge: $\langle dexter, alice, SELECT, 0 \rangle$ (Careful here!)
- ▶ When *alice* gave *bob* both read and insert privileges on *clients* we created additional two edges
- ▶ $\langle alice, bob, SELECT, 1 \rangle$, and
- ▶ $\langle alice, bob, INSERT, 1 \rangle$
- ▶ The question now is: Can *bob* insert into *clients*?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Here is the fundamental definition:
- ▶ Let p be a permission. Then the DBMS grants to a user u permission p on table t if the graph G_t has a path P from the node *system* to the node u satisfying the following conditions:
 1. All the edges on P are labeled with permission p
 2. All the edges on P , with the possible exception of the last one have the grant option bit equal to 1
- ▶ What does it tell us about permissions of user *bob* on *clients*?
- ▶ *bob* can not read (i.e. no SELECT) table *clients*, he also can not insert into it, nor update, etc.

Introduction

Few issues in
business, an aside

Access to data

Simple protection
mechanism - the
viewsAccess control in
SQLMandatory Access
ControlCompartment
mechanism, and
other access
modelsRole-based
access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ It should be quite clear that the graph of permissions for table t can be implemented as a table with the following four attributes: *grantor*, *grantee*, *permission*, *grant_control_bit*
- ▶ (We actually observed it a couple of slides above)
- ▶ Then, we can take union of all these tables, sort them according to the *table_name* and *privilege* and store it as one of *system tables*
- ▶ Let us call it *access*

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Finding if a user can get a privilege on a table is also simple; it is a variation on transitive closure (Floyd-Warshall) algorithm
- ▶ When a user *bob* wants to execute a privilege (say INSERT) on a table (say *clients*) then we find the part of the table access described above and process the corresponding graph

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Here is the corresponding SQL query:
SELECT *
FROM *access* WHERE *privilege* = INSERT
- ▶ The answer to this query describes the part of the graph $G_{clients}$ that deals with the privilege INSERT
- ▶ We need to divide this graph into two parts; the edges that have the control bit equal to 1, and the remaining part
- ▶ We compute the transitive closure of the first part

Introduction

Few issues in
business, an aside

Access to data

Simple protection
mechanism - the
viewsAccess control in
SQLMandatory Access
ControlCompartment
mechanism, and
other access
modelsRole-based
access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Then we do one more iteration; conjoin this transitive closure with the second part (why?)
- ▶ (Observe a relationship between join and composition of relations)
- ▶ Finally, we project on the *grantee* coordinate
- ▶ If *bob* is in that projection, the permission is granted, if not - it is not granted
- ▶ (Of course other implementations of DAC are possible, too)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Unexpected consequences of DAC model

- ▶ Say, *dexter* gave SELECT permission to *eve* before receiving one from *marek*
- ▶ Then, by granting permission to *dexter* to SELECT (with grant option) *marek* automatically gave *eve* permission to SELECT
- ▶ Thus the following “social engineering” scenario is possible: the user *eve* tricks the user *dexter* into giving her permission to SELECT by arguing that it changes nothing. Indeed it does nothing at the time of grant. But later on, when *dexter* forgot about the fact that it was done, he gets a permission to SELECT with grant option from *marek* and *eve* gets the access to the table *clients* without *dexter* realizing the consequences of previous actions
- ▶ Yet another variation on this theme is Trojan Horse attack (there are many Trojan Horses, this one is DAC Trojan Horse)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Once eve has a permission to SELECT, programs owned by her also have permissions to select (careful here!)
- ▶ So here is what eve does. She breaks into *marek* account just once and since *marek* left his password and database password on a yellow “sticky note” at the back of his monitor, she connects to the database just once and executes the following command:
- ▶ GRANT SELECT ON *clients* TO eve

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Now, eve writes a program that will copy the table *clients* every night at 2:00 a.m. to her account
- ▶ It is entirely legal from the point of view of DBMS (she has the permission to access, so her program also does)
- ▶ (And then she sells the contents of table *clients* on some *black hat* site)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ The examples show that DAC is susceptible both to “Social Engineering” attacks and also to “Trojan Horse” attacks
- ▶ Yet it has obvious advantages:
 - ▶ It has a simple semantics
 - ▶ It is implemented (i.e. supported by SQL)
 - ▶ It is convenient in business situations
- ▶ As long as the users know what they are doing it is relatively secure

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn?

CS 505- Spring 2014

- ▶ DAC is a flexible model of access control, well suited for business applications
- ▶ SQL supports DAC
- ▶ There are natural algorithms for access control under DAC
- ▶ DAC is *not* resistant to Trojan Horse attacks.

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What if we need more security?

CS 505- Spring 2014

- ▶ Security violations have their consequences
- ▶ I am not talking only about Government secrets, but ordinary businesses have their secrets, too
- ▶ I mentioned the issue of financial planning (share price targets, share splitting), product development, investment timing and many other issues
- ▶ In each of these cases someone with the access to internal information may get an unfair advantage
- ▶ In this process violations of law may occur
- ▶ Companies may be liable for such infringements
- ▶ Are there “stronger” models of security available?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Introducing Mandatory Access Control

CS 505- Spring 2014

- ▶ A mechanism to limit access to information so secrets can not be disclosed to unauthorized users
- ▶ This technique was pioneered by US Department of Defense in the 70es as so-called “Orange Book”
- ▶ It is called in the literature MAC, or “Bell - La Padula” model of security
- ▶ Over the years various aspects of MAC were modified and simplified
- ▶ The MAC security model treats database objects as “documents” which one can *read* or to which one can *write*
- ▶ An additional element of the MAC model is the set of *classifications*
- ▶ In this approach, there are possible various granularities of objects
- ▶ So objects could be tables, records, fields in records
- ▶ Let us fix the granularity to tables (but we do not have to)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ If you read the popular press, you know that US Gov't uses classifications: UNCLASSIFIED, CONFIDENTIAL, SECRET, and TOP SECRET
- ▶ But we could use any other terms. All that matters is that there is an ordering of the classifications. In the example above it is:

UNCLASSIFIED < CONFIDENTIAL <
< SECRET < TOP SECRET

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Likewise the users have classifications
- ▶ The classifications are kept as a separate part of the database, not accessible to ordinary users, in particular not modifiable by such users
- ▶ Observe that this did *not* happen in DAC!

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Of course, we can implement tables with classification as system tables
- ▶ But the owner of such table have to be the user *system* and a special mechanism would have to be used for protection of such tables
- ▶ Observe that we need *two* tables to represent classifications. One is for users, another for “documents” (in our case database tables, because of the granularity assumption we made above)
- ▶ The MAC model of security is based on two basic axioms:
 - ▶ Any user with classification c can write only to objects o that have classification t' such that $t \leq t'$
 - ▶ Any user with classification t can read only objects o that have classification t' such that $t' \leq t$

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ The classification of an object does not change during normal operation
- ▶ (That is we can only change classification of objects while they are not accessed)
- ▶ This protects the subjects from inadvertent incorrect reading/writing
- ▶ (There are subtler aspects of tranquility related to *need to know*: no one should have an access to information, until he needs it to do what she is supposed to do)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartments mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ User *marek* is classified CONFIDENTIAL, and document *doc1* is classified SECRET. User *marek* attempts to read *doc1*. The request is denied
- ▶ Document *doc2* is classified CONFIDENTIAL. User *marek* attempts to read *doc2*. The request is granted
- ▶ User *dexter* is classified SECRET. User *dexter* requests to write *doc2*. The request is denied
- ▶ But who grants or denies? a central control facility
- ▶ This facility must control and use in its operation access rights of all subjects and all objects, otherwise MAC technique fails

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Mandatory Access Control, cont'd

CS 505- Spring 2014

- ▶ The B-LP axioms are called “no write down”, and “no read up”
- ▶ The idea is that if you can not write down, then you will not divulge a secret you have to individuals (or their programs!) with lower classification
- ▶ Likewise, since you can not “read up”, you will not see data with classification that is bigger than yours, so you will not learn secrets above your “pay grade”
- ▶ Tables are assigned the classification of its creator, likewise application programs

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Mandatory Access Control, cont'd

CS 505- Spring 2014

- ▶ Now, we can prevent Trojan Horse attack because, say *marek* has classification SECRET, and so his table, *clients* has the classification SECRET
- ▶ Now eve has classification strictly smaller than SECRET (because otherwise she would have the access to the table *clients*, but she does not)
- ▶ So, eve has classification UNCLASSIFIED or CONFIDENTIAL
- ▶ When eve (or any program owned by her) attempts to read the table *clients* the request is rejected and DBA informed (wait till we talk about auditing)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Another side effect of MAC

- ▶ What if we go down in granularity, that is mark records, not tables for security levels?
- ▶ This requires an additional attribute “security level” (but this is a minor issue)

- ▶ Let us assume we have this table *bad_guys*

lname	fname	Class
smith	jack	S
thomson	jack	TS
morgan	tom	C

- ▶ *marek* has classification CONFIDENTIAL, *dexter* has classification SECRET
- ▶ Each of them writes a query: `SELECT * FROM bad_guys`
- ▶ What will *marek* see, and what will *dexter* see?
- ▶ Are we in the business of lying?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ This phenomenon of different users seeing different answers to the same query is called *polyinstantiation*
- ▶ When we go even lower with granularity (classifying attributes) this phenomenon leads to appearance of NULLs where we do not want to provide information
- ▶ Generally, the phenomenon of polyinstantiation is morally dubious

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ What should we do when the information is derived from classified information?
- ▶ For instance, medical records of, say HIV patients are protected. But what should we do when we compute derived data (for instance an average time from infection to the full-blown symptoms, or maximum levels of some proteins in blood)
- ▶ It looks like we no longer can derive information about individual cases
- ▶ Really?

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Here is how *some* information can be hacked if we are not careful (even so we do not allow for aggregate queries for small sets of records)
- ▶ I want to learn *marek's* salary (which is protected and I can not access it directly)
 - ▶ I ask for average salary over the large population
 - ▶ I ask for an average salary of everyone but *marek*
 - ▶ I ask for the size of population
- ▶ Clearly such data allows for computing *marek's* salary
- ▶ So, we need to be careful, and have policies controlling access to aggregates

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Is MAC a complete solution?

- ▶ MAC does not prevent attacks by “insiders”
- ▶ In particular, an insider may alter a table by writing to it (even if she can not read it!), with potentially serious consequences
- ▶ Generally (recall WikiLeaks affair) attacks by insiders are most dangerous
- ▶ So, additional means are needed
- ▶ Such means are called *compartments*
- ▶ Let us assume that we adopt the classification as above in a business situation
- ▶ There is a table *price_goals* containing temporal information on the expected prices of shares
- ▶ We realize that not only it is SECRET, but also that only people who are answering to CFO should have reading access
- ▶ So, an additional mechanism is needed

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartments mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ MAC pertains to operation systems, not only to DBMSes
- ▶ There are several dialects of Linux that provide MAC
- ▶ Also Solaris provides MAC
- ▶ In database world:
 - ▶ Oracle has an add-on called *Oracle Label Security* that implements MAC
 - ▶ MS SQL Server 2008 implements MAC
 - ▶ Public-domain PostgreSQL had a version supporting MAC

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

MAC and Orange Book model of security

- ▶ Generally, “Orange Book” called for separation of security module of database from the database itself
- ▶ It makes sense, but generally is *not* implemented in current generation of DBMSes
- ▶ Security is implemented using system tables within the DBMS itself
- ▶ Sometimes it is called *Reflexive Access Control*, but the same term is used for implementation of access tables for IP traffic in routers
- ▶ Since system tables are usually implemented as files in underlying OS, it creates the potential security problems (unless encryption is used)
- ▶ (Why?)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn?

CS 505- Spring 2014

- ▶ MAC is an access control strategy based on *centralized* control
- ▶ MAC requires a central facility to control access rights
- ▶ Users do not control access and they can not delegate
- ▶ But there is a price (possible polyinstantiation)
- ▶ Also we do not control enough of subjects (issue of compartments, that is *need to know*)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartments mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ In the situation as above, we create a *compartment* called *share_pricing*
- ▶ This gives us an additional mechanism for access control
- ▶ The users are marked with compartment bit
- ▶ Similarly, documents are marked with compartment bit
- ▶ The user *marek* can access a table which has compartment *share_pricing* if his bit corresponding to compartment *share_pricing* is 1, otherwise he can not access this table
- ▶ This mechanism is neutral w.r.t. security model (you can have MAC, DAC or other models with compartments)
- ▶ There may be many compartments, each with its separate marking bit

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartments mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Different access constraints require different policies
- ▶ (MAC, i.e. B-LP, was tailored towards protecting secrets)
- ▶ Biba policy is tailored towards preservation of data integrity (as opposed to secrecy)
- ▶ The idea is to make sure that data is modified *only* by individuals that are authorized to do so (notice that this was not the case in MAC)
- ▶ This is reflected by a policy opposite to B-LP:
 - ▶ No read down
 - ▶ No write up

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

More on Biba model

- ▶ The idea is that your authority (position within the business hierarchy) determines what you consider authoritative data
- ▶ For instance the dean *dexter* represents more authority than the departmental chair *alice*
- ▶ As they create “content”, this content is considered authoritative in their part of the organization, that is among people of the corresponding organizations
- ▶ Specifically, *dexter* writes authoritatively to all people in the college, while *alice* can only write authoritatively to people in the department
- ▶ (Observe that this model is used as a standard for distribution of instructions, policies, etc. in all hierarchical organizations)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn?

- ▶ There are “centralized” access control policies
- ▶ These include:
 - ▶ Bell - La Padula model
 - ▶ Biba model
 - ▶ Compartments

Introduction

Few issues in
business, an aside

Access to data

Simple protection
mechanism - the
views

Access control in
SQL

Mandatory Access
Control

Compartments
mechanism, and
other access
models

Role-based
access control

Chinese Wall

SQL injection

Encryption

Auditing

Role-based access control (RBAC)

- ▶ Deals with the business hierarchies
- ▶ Say, in a small bank, we have a table *accounts*
- ▶ Who can access this table?
- ▶ Certainly all tellers in that bank can access the table *accounts*
- ▶ Instead of giving access to the individuals who are tellers, we create a *role* called teller and give the permission to anyone having that role
- ▶ This simplifies and streamlines business process
- ▶ The reason is that a role may have a variety of privileges. It may have some privileges on one table and other privileges on another table
- ▶ So, when *dexter* is hired as a teller, all we need to do is to assign him this role and then he will automatically have various privileges on the table *accounts* but also (for instance) privileges on a table *clients*

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ In principle RBAC is neutral w.r.t. other access policies. We can have RBAC for DAC and RBAC for MAC
- ▶ A standard for RBAC was set by US NIST
- ▶ The SQL standards, starting with SQL-1999 support RBAC
- ▶ That is GRANT command allows for granting privileges to role and not only to individual users
- ▶ Also including users in roles is supported

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Role-based access control, cont'd

- ▶ Now, let us assume we are in a big bank, not a small bank
- ▶ Again we have tellers, but the security requires that *dexter*, who is a teller in Lexington accesses the accounts only of Lexington customers (a bit far-fetched example)
- ▶ This suggests a hierarchy of tellers. One role can be a *teller*, and also have a role *lex_teller*, with all *lex_tellers* being *tellers*
- ▶ This capability is implied by the business process
- ▶ Think any *lex_teller* can modify the table *lex_accounts*, but *lou_teller* can not do so, unless he is also in the role *lex_teller* (there is no reason why these roles be disjoint)
- ▶ Thus we can model all sorts of business rules with RBAC. For instance we may allow *some* tellers in Louisville to access Lexington accounts but not all such tellers

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Used in conflict-of-interest situations
- ▶ CW policy is sometimes called Brewer-Nash policy
- ▶ The idea is that besides the objects (like in B-LP) we also have *data sets*, collections of objects
- ▶ We assume that data sets form a partition of objects (think about collections of documents on specific clients)
- ▶ Then, we have a *conflict relationship* on the collection of data sets. This relation is supposed to be *irreflexive* and *symmetric*

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Here are the rules:
 - ▶ Access is granted to subject s on object o if either o is in a data set which s already accessed, or o is in a data set which is not in a data set which is in conflict with any of data sets accessed by s
- ▶ Observe that there is no difference for reading or writing
- ▶ The reason is that the user may read something that she should not, or write something she should not!

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

How are policies enforced?

- ▶ In DAC, generally on the DB server. The files used are system files that are stored on sme server
- ▶ In MAC, a security server is used to enforce the policy. It may be the same server, or not
- ▶ Similar issues arise with CW policy
- ▶ RBAC is neutral w.r.t. other policies (and is implemented on RDBMSes)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn?

- ▶ Besides of MAC, DAC there are other security policies
- ▶ All depends on the purpose of the access to data and its use
- ▶ Also, business model determines the access

Introduction

Few issues in
business, an aside

Access to data

Simple protection
mechanism - the
views

Access control in
SQL

Mandatory Access
Control

Compartment
mechanism, and
other access
models

Role-based
access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Say, we have within some code where we connect to database the following SQL command (which is a value of some variable):

```
SELECT * FROM accounts WHERE lname  
= "" + lname + "';"
```

- ▶ If I assign to variable *lname* the string *marek*, the records of all *mareks* will be returned

Introduction

Few issues in
business, an aside

Access to data

Simple protection
mechanism - the
viewsAccess control in
SQLMandatory Access
ControlCompartment
mechanism, and
other access
modelsRole-based
access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ But let us see what happens when I assign to *Iname* the string

`'OR '2' = '2'`

- ▶ Then I get the following: "SELECT * FROM *accounts* WHERE *Iname* = " OR '2' = '2' "
- ▶ The WHERE clause is evaluated as *True* (since $2 = 2$ is a true statement) and the *entire* table is returned

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ This may be used for attacking remote databases where the authentication is poorly programmed
- ▶ Imagine the following code for authentication:

```
myStr = SELECT uid FROM users where uid =  
'" & userId & "' AND pwd = '" pass + "'"
```

```
result = GetQueryResult (myStr)
```

```
If (result = "") Then
```

```
    auth = False
```

```
Else
```

```
    auth = True
```

```
End If
```
- ▶ Here *userId* and *pass* are taken from a Web form
- ▶ If I type as *userId*: ' OR “=‘
- ▶ and for *pass*: ' OR “=‘

Introduction

Few issues in
business, an aside

Access to data

Simple protection
mechanism - the
viewsAccess control in
SQLMandatory Access
ControlCompartment
mechanism, and
other access
modelsRole-based
access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ A funny thing happens
- ▶ The SQL command will return all tuples (because in the OR one of the disjuncts is (in words) empty equals to empty which is evaluated as True)
- ▶ Thus I will be authenticated, because nonempty string was returned!
- ▶ Worse things may happen too because SQL allows for executing several commands at once. So, if within my string I put something like

; DROP TABLE *accounts*;

- ▶ Then this may result in (temporary) destroying the table *accounts*, maybe even lost updates!

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ UNION operation (set-theoretic union of two tables) shares some properties with disjunction (because UNION amounts to disjunction of WHERE conditions)
- ▶ (I am a bit imprecise here)
- ▶ For that reason UNION was also used as a vehicle for SQL injection attacks
- ▶ Finally, attacks are not only getting the information out of the database or deleting records (or tables). One can maliciously insert (for instance making *marek* a criminal) etc.

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Defending against SQL-I

CS 505- Spring 2014

- ▶ Observe that if we keep the data in the table *users* encrypted, encrypt the inputs and then check, this specific attack will fail, because the result will not coincide with what is in database
- ▶ But we are not completely secure (why?)
- ▶ We can do more things, for instance check the types of inputs
- ▶ We can check for strings like DROP or DELETE (with variations)
- ▶ Of course attackers would try something like (this time in MySQL, because /* and */ are disregarded as is text in between): DR/**/OP TA/**/BLE accounts, etc.
- ▶ There is, fortunately, extensive literature on this topic
- ▶ The message is: *Be careful*, read literature on attacks and follow rules

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Some rules for fighting SQL-I

CS 505- Spring 2014

- ▶ (Taken from Ron Ben Natan book "Implementing Database Security and Auditing")
- ▶ All data needs to be sanitized of any string that should not be part of the input
- ▶ Never use string concatenation in SQL
- ▶ Prepared statements should be used as much as possible (because then the SQL strings are different from the values obtained from the user)
- ▶ (You may recall from CS405 that a prepared statement is a specific SQL template into which you interpolate some quoted parameters, so user input is *not* interpreted as SQL)
- ▶ Application login should be implemented within well-validated stored procedure
- ▶ Encrypt, if the application is critical (because encryption transforms data)
- ▶ Use quotes on all parts of user input (so the input is not treated as SQL)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn?

- ▶ SQL Injection is a *real danger*
- ▶ Database information may be altered or lost as the effect
- ▶ But we are not defenseless

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Often encryption is used in access control
- ▶ For instance password may be send encrypted
- ▶ In secure applications encryption may be use “end-to-end”, i.e. entire communication with database may be sent encrypted (including sending back the answers to queries)
- ▶ In fact, e-commerce is possible only because such capabilities exist
- ▶ Encryption is either public-key or private-key; public key encryption is expensive and is used to exchange a private-key (for a session)
- ▶ More on this topic availability in Cryptography books/lectures

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ “Man-in-the-middle” attack
- ▶ “Replay” attack
- ▶ Various attacks associated with networking (e.g. denial-of-service)
- ▶ Need for actors beyond sender and recipient
- ▶ Need for public-key infrastructure

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn?

- ▶ Connecting by remote users open ways to a variety of attacks
- ▶ While we are not defenseless, attacks are a frequent reality
- ▶ Various applications (esp. e-commerce) are susceptible to such attacks

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ General principles of access to DBs are similar but not identical to those that you (should have) learned at your Operating Systems course
- ▶ The general principle of access to databases is that *There should be no unauthenticated access*
- ▶ This is sometimes called *C2 security*, after US DoD book on security (the “Orange Book”) which specified “levels of security”
- ▶ “Orange Book” was a 1980s technology. The modern specifications of terminology and levels of security can be found in the document called “Common Criteria”, a joint venture of US Government (DoD, NIST) and European Union
- ▶ “Common Criteria” specify:
 - ▶ What are “evaluation criteria levels” (EAL)
 - ▶ What need to be tested for each level
 - ▶ Who can test
 - ▶ Who certifies testers

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ A basic requirement is that there must be *audit trail* for login to and logout from the DBMS
- ▶ Generally, maintained as a table with time, name of the user, sessionID, user name, which database is being used
- ▶ (Similar audit trail is also used for logins on the system)
- ▶ It is easy to implement such audit trail with triggers, but specialized add-on's for DBMSes are available
- ▶ Of course one thing the intruder will try to do is to alter the audit trail, so it must be kept secure, possibly on another machine not directly accessible from the machine that has audited DBMS

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Audit, cont'd

- ▶ What else needs to be audited depends on the type of applications
- ▶ We may want to know how much each user uses the database, in terms of number of sessions, of space used, of programs that are being executed
- ▶ An important aspect of usage is the time periods when the database is used; any usage outside of “normal operating hours” should be considered suspicious
- ▶ Sometimes we want to know data-definition activity (like: did a new table or view was defined?)
- ▶ The point is that such activity may be not permitted by law (say, privacy laws)
- ▶ For instance, in various countries of Europe customer data can be kept maximum of 6 months, so copying of that data into a new table may lead to a violation of local laws. There is *no* international standard for keeping customer data.

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Accounts w/o passwords need to be purged
- ▶ Quality of passwords must be audited
- ▶ Who created accounts?
- ▶ Database errors need often be audited (because they, for instance, can indicate SQL injection attacks)
- ▶ Since triggers and stored procedures may do unexpected things, changes to such procedures may need to be audited

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

- ▶ Changes to privileges may need to be audited, because such changes may indicate presence of the attack (possibly internal jobs)
- ▶ Changes to roles may need to be audited because they may result in integrity loss (e.g. suddenly user *marek* who is, actually, a janitor gets into a role *teller*)
- ▶ Data that is deemed sensitive should be audited (for instance user *marek* may try to update his personal record, giving himself an unjustified raise)
- ▶ SELECT statements may need to be audited for privacy (for instance recall the Ohio gov't workers searching state-owned databases for information about "Plumber Joe", or State Department workers searching passport files of celebrities)

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

Audit, cont'd

- ▶ Dormant accounts are dangerous and so we need to audit them
- ▶ It is beneficial if auditing systems are external to DBMS (although all major DBMS software provide audit facilities)
- ▶ In fact all major business software companies (including SAP, IBM and others) provide audit systems
- ▶ Auditing information should be properly archived because some auditing is required by law (HIPAA – healthcare, Gramm-Leach-Bliley Act – financial services, Sarbanes-Oxley – publicly held companies, California law SB 1368 – maintenance of personal information)
- ▶ Use of encryption is beneficial because it proves persistence
- ▶ Auditing of what is audited may be needed

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing

What did we learn?

- ▶ Without a well established audit procedures DBMS is insecure
- ▶ DBA (or in larger places, IT department) is responsible for audit
- ▶ Audit is not a one-time activity but a continuing effort

Introduction

Few issues in business, an aside

Access to data

Simple protection mechanism - the views

Access control in SQL

Mandatory Access Control

Compartment mechanism, and other access models

Role-based access control

Chinese Wall

SQL injection

Encryption

Auditing